

DATE CAM-monitoring paper – Indranil - Focus Areas

1. Matched cell-count experiment (do this first): Right now the paper compares the ternary and thermometer codes at the same number of *bits* (384), and ternary wins cross-domain (average 0.640 vs. 0.579; 63 of 90 cells). That said, a ternary cell may cost about twice the hardware of a binary cell, so the fair comparison gives thermometer twice the code length. Re-run the ten-domain grid with the thermometer code at roughly 768 bits, same splits. A reviewer will ask for this, and the result decides which encoding we lead with in Sec. V-C. *Send back:* the diagonal and off-diagonal means, and the cell-by-cell win count against ternary at 384 b.

IB: Thermometer at 768 b (256 directions $\times$ 3 bits), same splits as the 384-b runs: in-domain mean 0.924, cross-domain mean 0.631. Against ternary at 384 trits (0.920 / 0.640): cross-domain mean delta -0.009 ; ternary still ahead in 54 of 90 off-diagonal cells (was 63 of 90 at 384 b). Worst off-diagonal cell improves from 0.259 (ternary) to 0.319. So at matched cell count the two are within noise on the mean, with ternary keeping a small cell-wise edge. Files: `bits_grid_768b_L4_m256.csv`, `matched_cell_count.py`.

2. Combined-training score: The source paper’s strongest result for keeping the library small is its “combined” row – one probe trained on all of their datasets pooled, which scores well everywhere (0.97 even on sycophancy). Our grids have no combined row. Build a bank from all ten domains pooled and score it on each of the ten test domains, for both 384-b encodings and the full-precision ceiling. This matters because the combined row is the best case *against* our large- K argument, and we should report our own numbers for it rather than argue around theirs (see objection (1) in the appendix document). *Send back:* one ten-value AUROC row per encoding, plus the ceiling’s.

IB: Pooled bank (all ten domains’ train splits, 11,278 rows), AUROC per test domain in the order [defn, empir, fict, logic, ethical, syco, repe, roleplay, insider, sandbag]:

thermometer-384: 0.936 0.976 0.966 0.803 0.977 0.627 0.896 0.887 0.997 0.998 (mean 0.906)

ternary-384: 0.937 0.976 0.964 0.747 0.977 0.635 0.936 0.919 0.997 1.000 (mean 0.909)

cosine ceiling: 0.970 0.996 0.983 0.803 0.983 0.636 0.957 0.941 0.997 1.000 (mean 0.927)

Pooling costs nothing against the per-domain banks (diagonal means 0.919 / 0.920 / 0.928), but sycophancy stays at 0.63 – unlike their combined probe’s 0.97. File: `combined_bank.csv` (`combined_bank_auroc.py`; the file also has a leave-one-domain-out row, bank = other nine domains).

3. Fig. 2(d)–(f) data and names: Confirm the data behind panels (d)–(f) is the current run – this slipped once before. Also check our domain names against the source paper: they call their five truth types definitional, empirical, fictional, logical, and ethical; our grids say “evidential” and “ethics.” If ours are the same datasets, relabel to match; if not, tell me what they actually are. Same check for roleplay vs. roleplaying and insider_trd vs. insider trading.

IB: Panels (d)–(f) checked cell by cell against the result files: all three are the current 384-b runs (seed 0, 70/30 split). Names: our evidential is their empirical, and our ethics_commonsense is their ethical (same ETHICS-commonsense source), so both are the same datasets. Relabeling: evidential $\rightarrow$ empirical, ethics $\rightarrow$ ethical, roleplay $\rightarrow$ roleplaying, insider_trd $\rightarrow$ insider trading. repe_honesty stays as is since it isn’t one of theirs. In the figure, panels (d)–(f) “evid” will be replaced by “empir” to match the top row.

4. Ternary dead zone: Our “ternary” code maps small projection values to don’t-care. Confirm the dead zone was actually turned on in the 384-b runs. If it was off, the code was binary in practice, and the cell-type claims in Secs. IV and V and Table II need to change.

IB: Confirmed on. The grid script only has a ternary encoder (no binary option), and it maps the smallest 20% of projection magnitudes in each column to don’t-care by default. The 384-b ternary

grid is genuinely ternary, and the TCAM-cell claims in Secs. IV and V and Table II stand as written. One detail: the don't-care threshold for queries is computed from the test batch rather than fixed from training; I will fix it to use a training-set threshold and rerun to confirm the numbers hold.

5. Audit at 384 b: The destroyed-vector audit numbers in Sec. V-E (mean 0.50; 83% of cells between 0.4 and 0.6) come from the 2,048-b grid. Run, or locate, the same audit at 384 b. If we only have it at 2,048 b, say so and the text will state that instead.

IB: At 384 b the audit collapses to mean 0.500 for ternary (all 100 cells within 0.40–0.56) and 0.501 for thermometer (all cells within 0.44–0.57). Both from the label-shuffle audit. The “0.50 / 83%” now in Sec. V-E is from the 2,048-b run; replace it with these.

6. Which four-domain numbers are current: The paper quotes the undergrads’ deck (average gap -0.03 , worst cell -0.09); the Aug. 14 methods document said -0.06 / within 0.05. Tell me which run is current, so the paper quotes one set of numbers.

IB: Those four-domain numbers aren’t from my runs. The reference values (1.00/1.00/1.00/0.95) are the Truthfulness Spectrum paper’s own in-domain results for definitional, empirical, fictional, and logical; the measured values and the gaps (-0.03 / -0.06) come from the undergrads’ first pipeline. All my runs are on the ten-domain grid, so I can’t say which of their runs is current.

7. The decision rule – walk me through it: Sec. IV-E says we store a few thousand labeled example activations per domain and classify a query by a majority vote of its 31 nearest stored rows. My note on the draft flagged that I am not sure this matches what the runs actually do. Walk me through what is stored and what is compared. If it is not thousands of exemplars with a $k=31$ vote, then that subsection, the audit discussion, and Table II’s bank sizes all need to change.

IB: For each domain I take the 70% training split, run every example through the frozen encoder (mean-center, random projection to 128 dims, quantize to 4 levels, thermometer coding), and store the resulting 384-bit code together with its honest/deceptive label. So the bank really is thousands of rows per domain, 831 for definitional up to 3,201 for sycophancy.

At query time a test activation goes through the same encoder with the same fitted mean, projection matrix, and cutoffs, and its code is compared by Hamming distance against every stored row in the bank. I take the 31 rows with the smallest distance and compute the fraction of them labeled deceptive; that fraction is the score for the query.

The score is a number between 0 and 1, not a yes/no. AUROC is computed directly on that number, so I never pick a threshold or take a majority. The bank description and Table II’s row counts are correct; the only imprecise phrase is “majority label” – it should say “the fraction of deceptive neighbors.”

8. If time remains: (a) The recombination experiment: a frozen random projection, quantized to 3 bits, matched on a Euclidean matchline, at matched bank size on the same grid. A few days of work, and it settles whether the Euclidean and Hamming tracks are two paths or one. (b) A tuned GPU baseline (GEMV/FAISS) at batch 1 and batch 64, for Sec. VII’s estimates.

IB: (a) I kept everything the same as the existing grids: same splits, same frozen random projection to 128 values, same $k=31$ vote and only changed the distance used to compare rows. Each of the 128 values was rounded to one of 8 levels (3 bits). Then I scored the same grid three ways: Hamming distance (what a binary CAM does; 896 cells per row), Euclidean distance (what a multi-bit cell does; 128 cells per row), and Euclidean on the unrounded values as a reference.

Results: Hamming 0.920 in-domain / 0.593 cross-domain; Euclidean 0.920 / 0.579; unrounded Euclidean 0.916 / 0.598. So Euclidean is slightly worse than Hamming cross-domain (0.014, ahead in only 37 of 90 cells), and at 4 levels the two are equal (0.003). Rounding to 3 bits costs about 0.02.

Takeaway: once the projection is frozen, the multi-bit cell gives no accuracy advantage over a plain

binary CAM, its only benefit is fewer cells per row. The accuracy edge of the first pipeline came from its trained projection, not from the Euclidean matchline.

(b) Two GPU baselines on an H100 PCIe, each including the $k=31$ top-k, at K up to 131K (the whole-vocabulary bank) and batch 1 and 64, with board power sampled during the timed loop.

First, the raw fp16 bank (the one Eq. 2 prices): at 131K rows, batch 1 takes 1.22 ms and 277 mJ per token; batch 64 takes 21 μ s and 4.7 mJ per token. Achieved bandwidth was 1.6–1.8 TB/s, about half of spec, so the $K \sim 204K$ crossing in Sec. III is closer to 107K in practice.

Second, the same 384-bit compressed codes the CAM holds, searched on the GPU with FAISS’s popcount kernel: at 131K rows, batch 1 takes 28 μ s and 2.1 mJ per token (0.45 mJ above idle); batch 64 takes 0.5 μ s and 0.04 mJ per token. Latency is flat from $K = 10^3$ to 131K, so the scan itself is under 30 μ s and the GPU sits at 3 percent of its per-token budget at 1,000 tok/s.